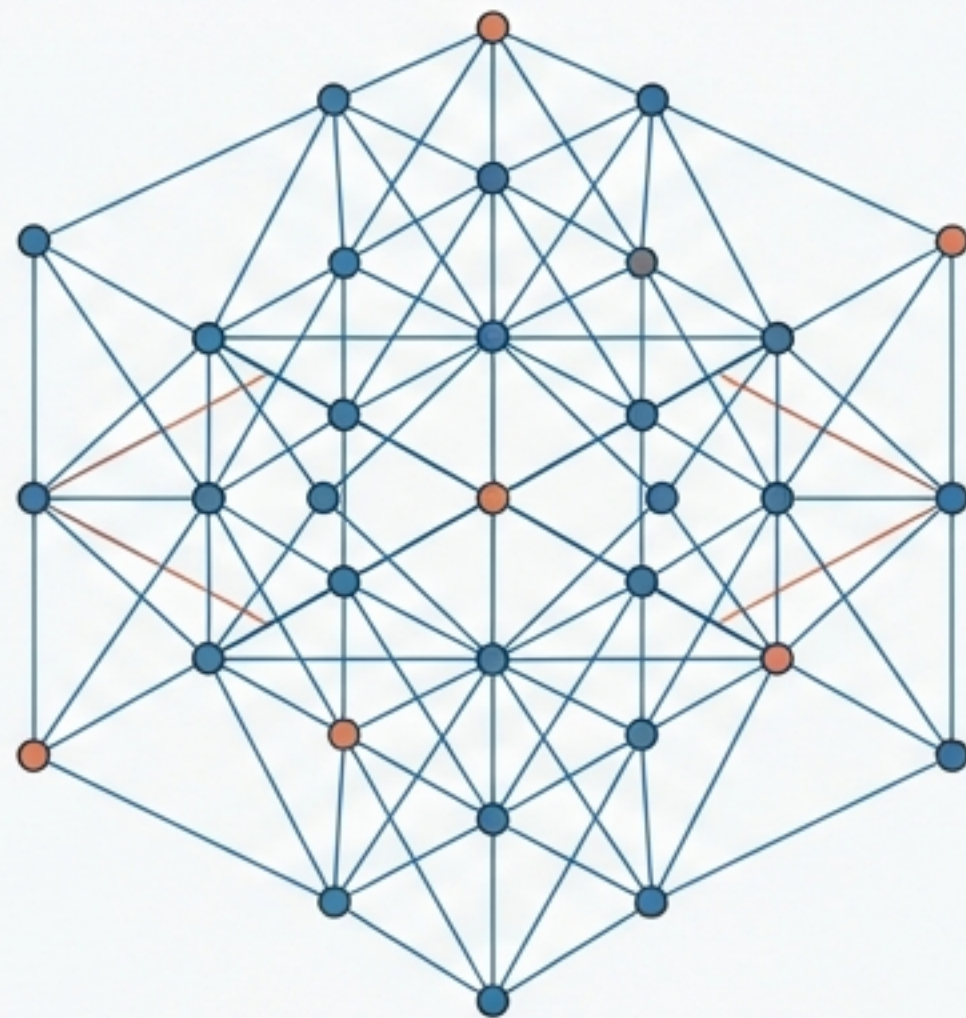


MGraph Cache Service: Storage Operations & Architecture



Technical Brief v1.4.45
Service: mgraph-ai-service-cache

Reference: Html_Cache_Client, FLeT Architecture v1.4.44

An Architecture Built for Scale and Speed

The MGraph Cache Service implements a Two-Tier Storage Model, strictly separating Entity Management from Data Storage.

Deduplication: Hash-based strategy prevents redundant storage of identical URLs.

O(1) Lookups: Dual indexing (By-Hash and By-ID) ensures immediate retrieval regardless of dataset size.

Flexible Organization: The 'Data Folder' concept allows multiple output types (HTML, Graphs) to sit under a single identity.

High-Level Architecture

Tier 1: Identity



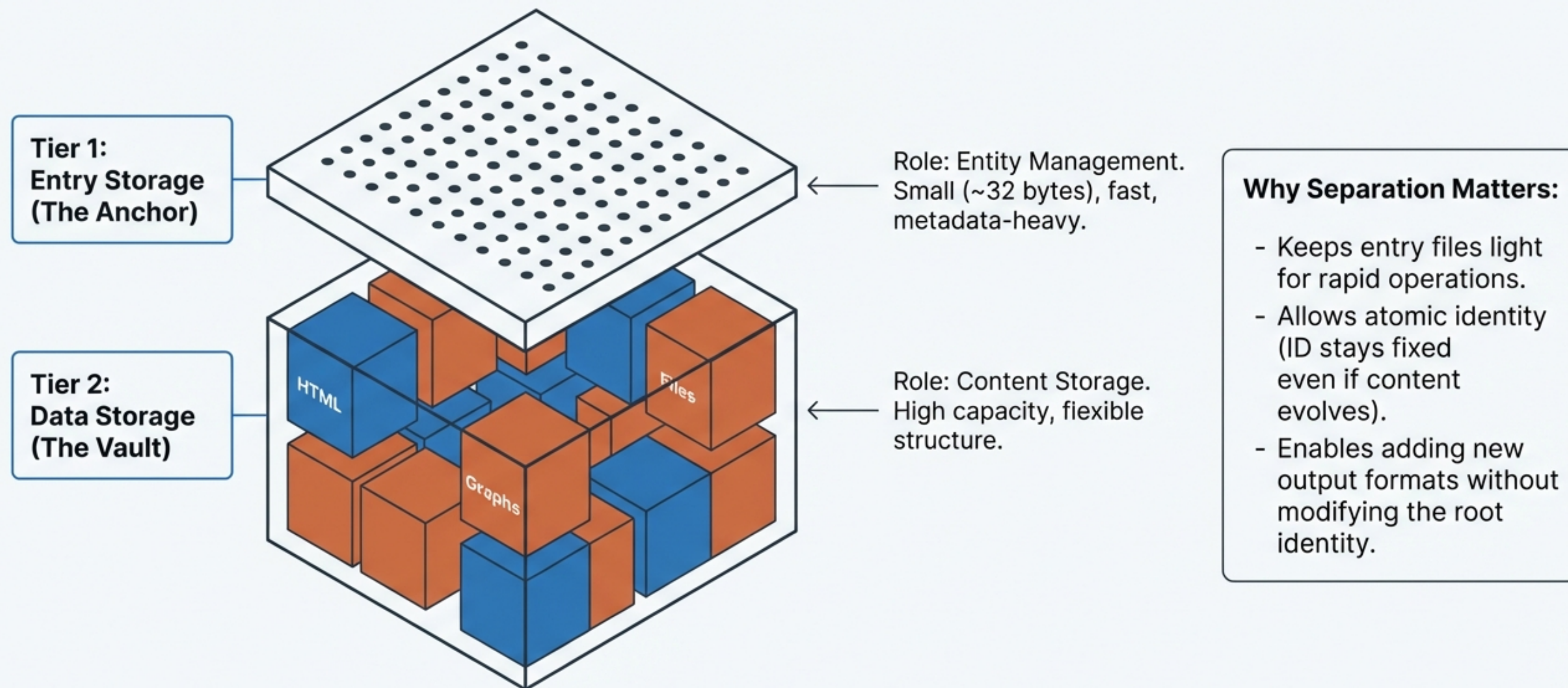
Tier 2: Content



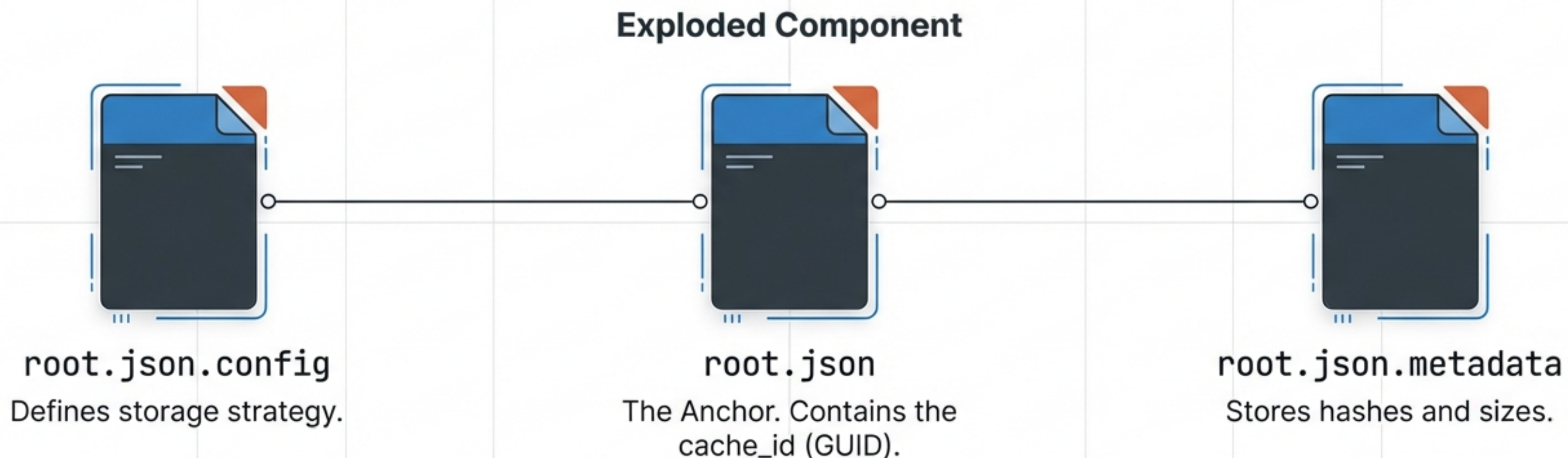
Navigating the Complexity of Web Content Storage

	Challenge	The Friction	The Solution
	Deduplication	How to ensure the same URL isn't stored twice?	Hash-based unique constraints.
	Identity	Uniquely identifying an entry amidst dynamic changes.	Atomic GUIDs.
	Lookup Speed	Checking "do we have this?" without latency.	O(1) Indexing.
	Data Organization	Managing raw vs. clean vs. graph outputs for one URL.	Dedicated Data Folders.
	Auditability	Tracking provenance and changes.	Immutable Metadata chains.

The Solution is a Two-Tier Storage Model



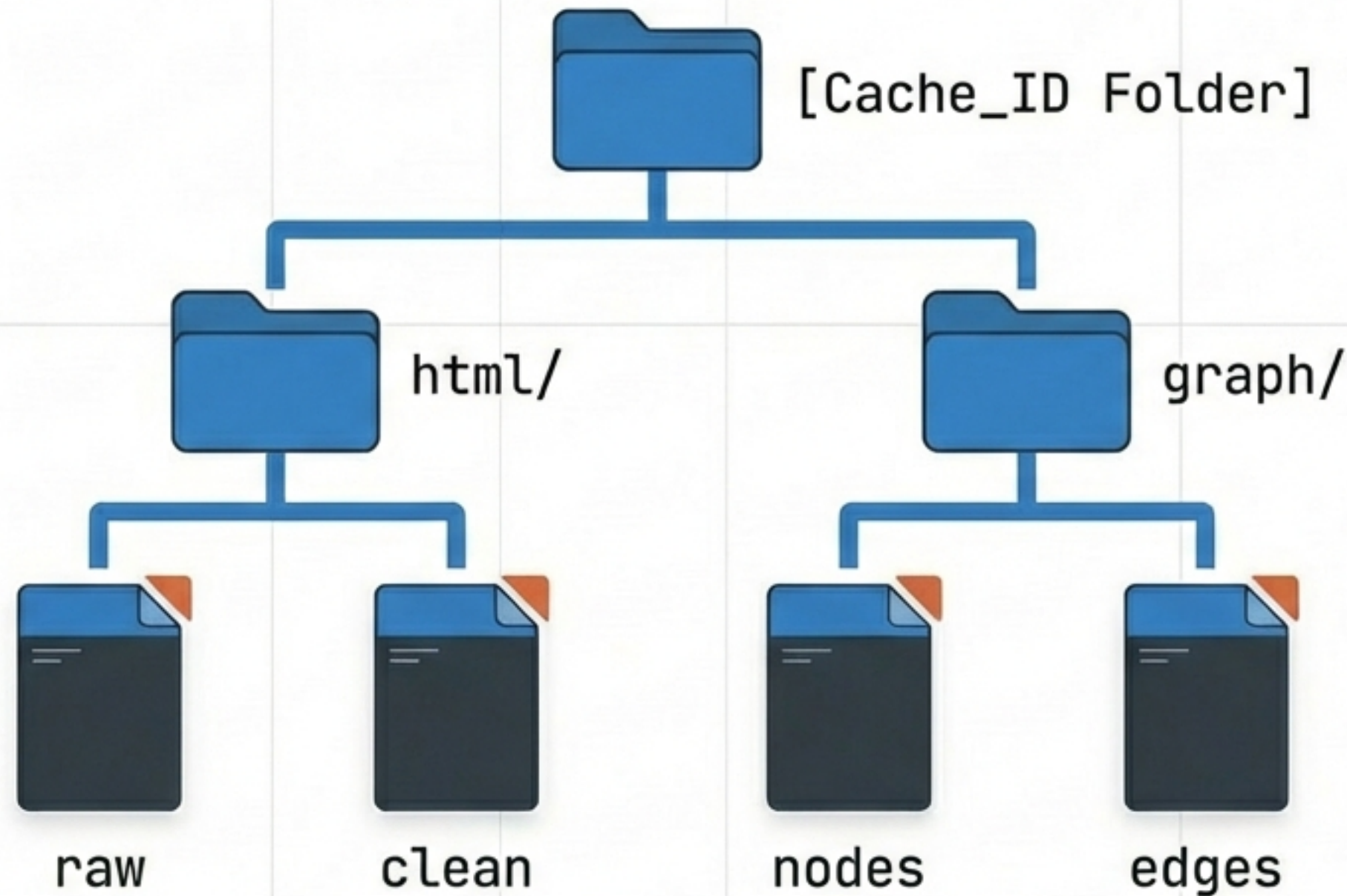
Tier 1: The Anchor and Identity



Key Insight: The entry file essentially declares 'I exist' and provides the coordinates to find the actual substance.

Tier 2: The Data Folder Hierarchy

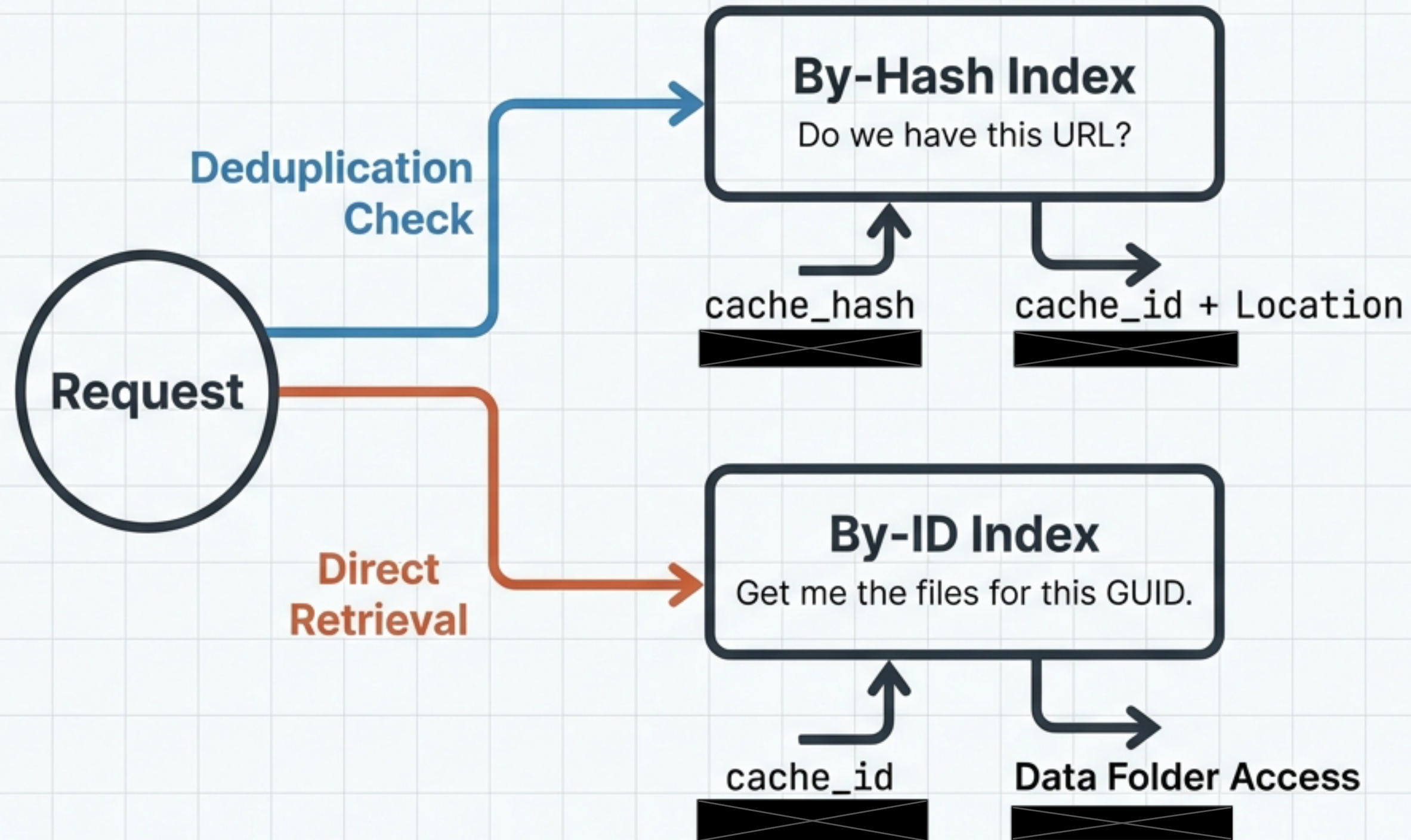
One Identity, Multiple Outputs.



The Data Folder Concept:

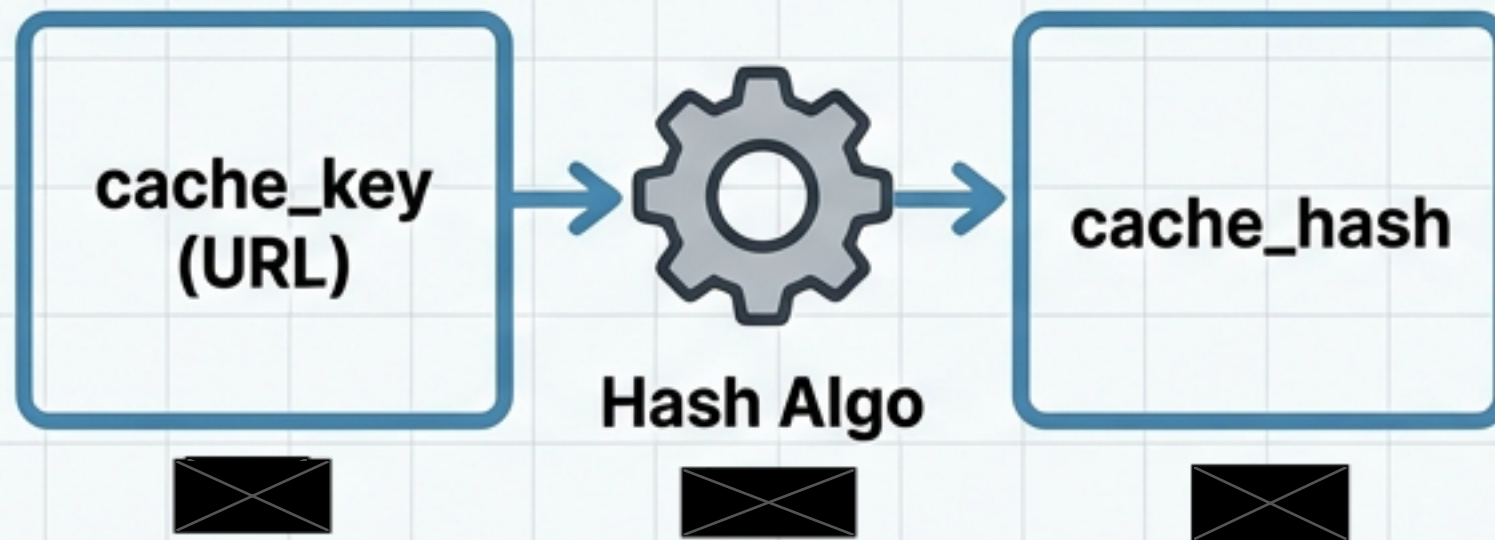
- A dedicated directory for every unique entry.
- **data_key**: Path within folder (e.g., 'html/').
- **data_file_id**: Filename (e.g., 'raw').

The Dual Reference System



Hashing Strategy and Deduplication

Identity Hash



Example:

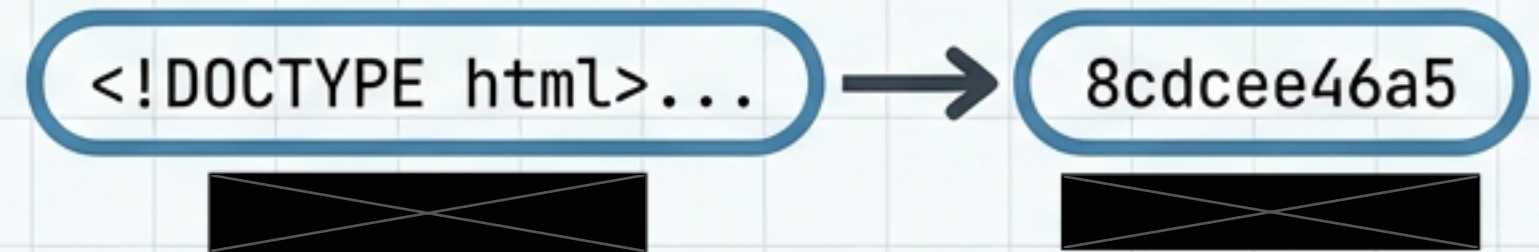


Purpose: Deduplication. Prevents storing the same URL twice.

Content Hash



Example:



Purpose: Change Detection. Determines if page has changed since last cache.

Anatomy of the File System

EXPLORER

```
namespace/ (test-html-to-cache)
├── 3f/ (Shard 1)
│   └── e6/ (Shard 2)
│       ├── a1b2c3d4-e5f6.../ (Entry ID)
│       │   ├── root.json
│       │   ├── root.json.config
│       │   └── root.json.metadata
│       └── data/
```

2-Level Sharding Strategy

Prevents directory bloat. Handles millions of entries while keeping listings under ~4000 files.

Metadata and Auditability

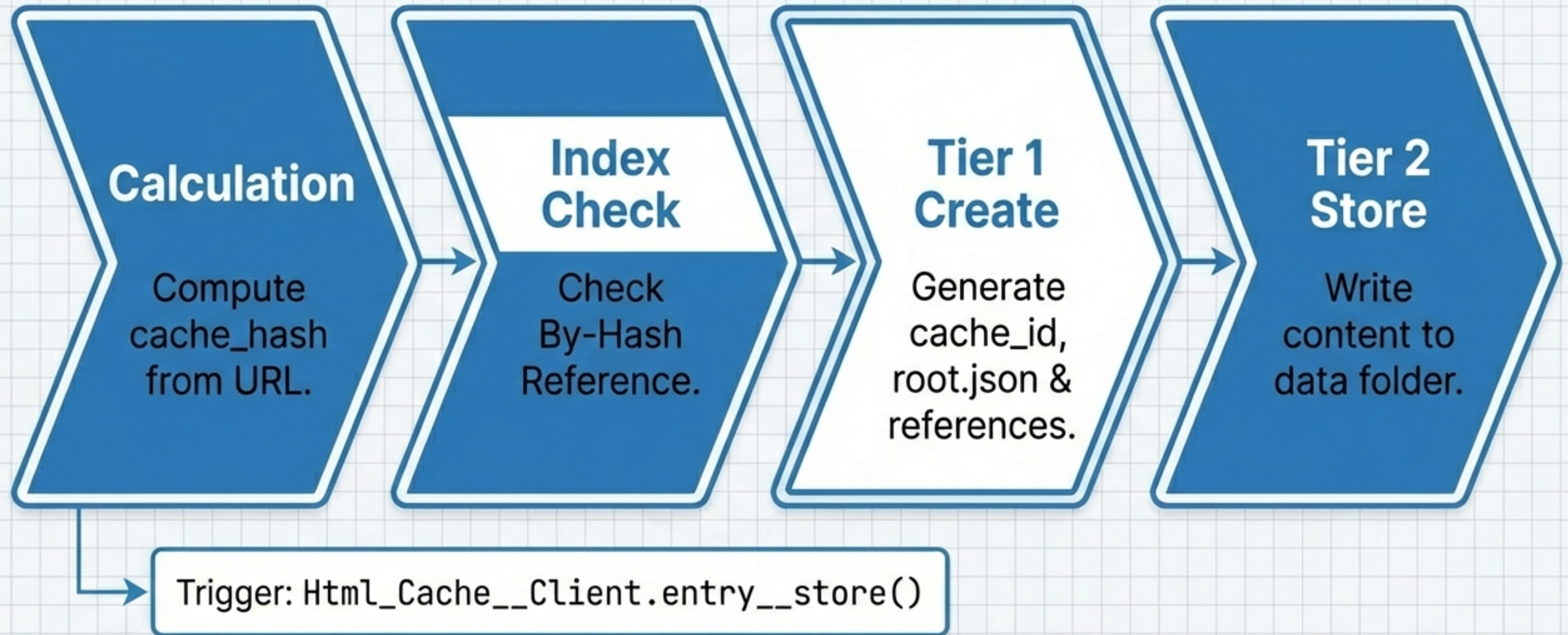
```
{  
  'content__hash': '8cdcee46a5...',  
  'content__size': 1024,  
  'data': {  
    'cache_hash': '3fe62295e6...',  
    'json_field_path': 'cache_key'  
  },  
  'timestamp': '2026-01-15T12:00:00Z'  
}
```

Detects content
drift/changes.

Links back to the
original URL identity.

Audit history.

The Storage Sequence: Life of a Request



Developer Reference and Implementation

```
VS Code IDE

class Html_Cache_Client:
    def entry__store(self, <
        cache_key: str,
        namespace: str,
        content: str):

        # Calculates hash
        # Checks index
        # Writes Tier 1 & Tier 2
        pass

    def data__store_string(self, <
        cache_id: str,
        data_key: str,
        data_file_id: str):

        # Updates Tier 2 only
        pass
```

→ **entry__store():** Primary entry point.

→ **data__store_string():** Used by FLeTs for content.

→ **cache_key:** Semantic identifier (URL).

Diagnostics and Troubleshooting

Cheat Sheet

Symptom	Check / Diagnosis
Lookup Failure	Check if cache_hash matches current cache_key. Verify 'By-Hash' reference file exists. 
Version Drift	Compare content_hash in metadata vs. runtime calculation. 
Diagnostic Query: Trace the cache_key → cache_hash → cache_id flow to find broken links.	



System Architecture Summary

